

Reporte de Mejoras de Seguridad y Migración a Validación Server-Side

Rediseño orientado a un modelo de confianza depositado exclusivamente en el servidor (server-side trust).

Preparado para: Equipo de Desarrollo e Infraestructura

Fecha de Emisión: Febrero, 2025

Estado del Documento: Propuesta Técnica de Rediseño

Origen del Análisis: Auditoría de Seguridad Arquitectónica

1. Problemas Fundamentales Detectados

Tu sistema actual depende de manera excesiva de lógica ejecutada del lado del cliente. Esto le permite a un atacante potencial:

- Reproducir algoritmos de atestación de forma offline y controlada.
- Interceptar, descifrar y modificar el tráfico de WebSockets en tiempo real.
- Parchear loaders y ejecutables binarios directamente en memoria.
- Reemplazar verificaciones criptográficas HMAC locales mediante inyección.
- Automatizar desvíos (bypasses) multiplataforma emulando respuestas.

La principal debilidad identificada radica en que los secretos criptográficos se pueden derivar directamente analizando tus archivos de distribución pública.

2. Objetivo del Rediseño

Tu objetivo principal tiene que ser migrar de un esquema basado en 'cliente confiable' a un modelo que asuma las siguientes premisas estructurales:

- **Cliente no confiable (untrusted client):** se asume siempre el peor escenario del lado del dispositivo local.
- **Autoridad absoluta del servidor:** el backend es la única entidad que define el estado real y las transiciones válidas.
- **Validación basada en secretos no exportables:** claves temporales que no residen en el almacenamiento local de manera persistente.
- **Detección de comportamiento:** priorizar el análisis conductual constante por sobre la mera validación binaria estática.

3. Eliminación de Secretos Embebidos

No debés derivar ninguna clave permanente a partir de los siguientes elementos del sistema de archivos local:

- Tamaños o firmas hash de archivos en disco.
- Offsets o rangos de direcciones de memoria predecibles.
- Assets o archivos multimedia descargados dinámicamente desde tu CDN.

Toda clave crítica se tiene que originar de manera exclusiva en tu backend, usando entropía robusta, y debe expirar en plazos sumamente cortos.

4. Remote Attestation Real

Te sugerimos implementar las siguientes prácticas de atestación remota:

- **Firmas asimétricas:** autenticación robusta mediante par de claves pública/privada.

- **Certificados efímeros:** emitir credenciales válidas únicamente durante el tiempo de vida de la sesión activa.
- **Uso de hardware seguro:** evaluar soporte para TPM o Secure Enclave cuando el dispositivo de destino del usuario lo permita.
- **Desafíos dinámicos (challenges):** validar la integridad mediante retos del servidor que resulten físicamente imposibles de precalcular de forma estática.

La atestación nunca tiene que depender de constantes estáticas compartidas entre distintas versiones del cliente.

5. Migración de Validaciones al Servidor

Tu servidor tiene que asumir que el entorno del cliente puede estar completamente controlado por un tercero. Te recomendamos delegar al backend las siguientes validaciones:

- Comprobación estricta de movimientos o velocidades imposibles según la física definida.
- Control de tasa de peticiones y límites de velocidad generales (rate limiting).
- Reglas autoritativas para la gestión de transacciones e inventario del usuario.
- Simulación de lógicas críticas de forma paralela en el servidor.
- Validación rigurosa de tiempos de espera (cooldowns) y flujos económicos.

6. Protección de Assets

Podés mantener técnicas de ofuscación de código como una barrera de costo económico frente al análisis técnico, pero bajo ninguna circunstancia tenés que considerarla tu mecanismo de seguridad principal. Te sugerimos:

- Rotar los assets de forma frecuente durante actualizaciones rutinarias.
- Segmentar de forma estricta los paquetes de recursos según el perfil o progreso del usuario.
- Incorporar marcas de agua digitales vinculadas temporalmente a la sesión activa.
- Validar firmas digitales remotas antes de cargar recursos pesados en el cliente.

7. Seguridad de Red

Sugerimos aplicar estas mejoras en la capa de transporte:

- Configurar TLS de manera estricta, limitando el uso a versiones modernas de protocolo.
- Implementar mecanismos de certificate pinning en el canal de datos principal.
- Desarrollar rutinas de detección contra proxificación local (MITM).
- Utilizar tokens portadores de corta duración y revocación inmediata en caso de anomalías.
- Firmar la telemetría usando claves simétricas negociadas de forma única por conexión.

8. Detección Conductual

Los esquemas modernos de protección se apoyan más en el análisis de comportamiento que en el intento de ocultar algoritmos en el binario. Evaluá la incorporación de telemetría para identificar:

- Patrones de entrada que violen la velocidad de reacción humana o la precisión mecánica.
- Sincronización exacta de acciones que denoten automatización de procesos.
- Desvíos de latencia inconsistentes respecto del comportamiento histórico de la conexión del usuario.
- Análisis estadístico agregador que compare desvíos de variables estándar.

9. Arquitectura Recomendada

Te sugerimos adoptar una arquitectura autoritativa estructurada bajo el siguiente flujo lógico de validación:

Cliente → Gateway Seguro → Servicios Autoritativos → Motor de Validación

Bajo este modelo se garantizan las siguientes reglas:

- Tu cliente no retiene claves permanentes en ningún momento.
- El Gateway seguro actúa como barrera de acceso y emite credenciales efímeras por sesión.
- El motor de validación se ejecuta en el backend y valida las acciones críticas sin requerir confirmación local.

10. Conclusión

El principal inconveniente del diseño actual radica en asumir que un cliente distribuido al público general puede operar de manera confiable como raíz de confianza criptográfica.

En un escenario real donde el atacante posee acceso al binario compilado, controla el direccionamiento de memoria de su dispositivo, puede interceptar interfaces físicas de red y ejecutar ingeniería inversa, tenés que estructurar toda la arquitectura asumiendo el compromiso total del extremo del cliente. La seguridad sostenible se construye desde el servidor, eliminando la confianza en el software cliente.

Tabla Comparativa de Esquemas

Esquema Actual (Vulnerable)	Rediseño Propuesto (Server-Side Trust)
Constantes derivables en disco/código	Secretos efímeros generados en backend
Verificación HMAC local	Firmas criptográficas validadas en el servidor
Validación lógica delegada al cliente	Servidor como única autoridad lógica
Cifrado basado en operadores XOR estáticos	Criptografía de sesión asimétrica y dinámica
Dependencia estricta de anti-tamper local	Análisis de comportamiento y telemetría remota